

AntroPy: Spectral Entropy

This example demonstrates how to calculate **spectral entropy** using **AntroPy** (`spectral_entropy`).

The workflow follows the EEG Feature Atlas structure:

Signal → **Method** → **Feature** → **Numerical Output** → **Visualization** → **Interpretation**

We will:

1. build intuition with two synthetic signals (clean vs. noisy);
2. calculate spectral entropy for both;
3. download a public EEG recording;
4. select a single EEG channel and calculate its spectral entropy;
5. compare all three values numerically and visually;
6. interpret the feature carefully;
7. summarize the feature for the Atlas.

Library / Method / Feature

Field	Value
Library	AntroPy
Function	<code>ant.spectral_entropy()</code>
Method	Shannon entropy of the power spectral density (PSD)
Feature	Spectral entropy (complexity measure)
Domain	Spectral / Signal complexity
Input	1-D signal (synthetic array or single EEG channel)
Output	Single scalar value
Visualization	Bar chart (comparison across signals)

Important distinction:

- PSD estimation (via `method='welch'` or `'fft'`) = spectral estimation step, internal to the function;
- Spectral entropy = the feature extracted from that PSD;
- Bar chart = visualization used to compare the resulting scalar values.

Spectral entropy is defined as the Shannon entropy of the normalized PSD:

$$H(x, sf) = -\sum P(f) \cdot \log_2[P(f)]$$

where P is the normalized PSD and the sum runs over all frequencies from 0 to $sf/2$. When `normalize=True`, this value is divided by $\log_2(N)$, where N is the number of frequency bins, so the result is bounded to **[0, 1]** regardless of signal length or sampling rate.

Part 1 — Building intuition with synthetic signals

Spectral entropy measures how concentrated or spread out a signal's power is across frequencies.

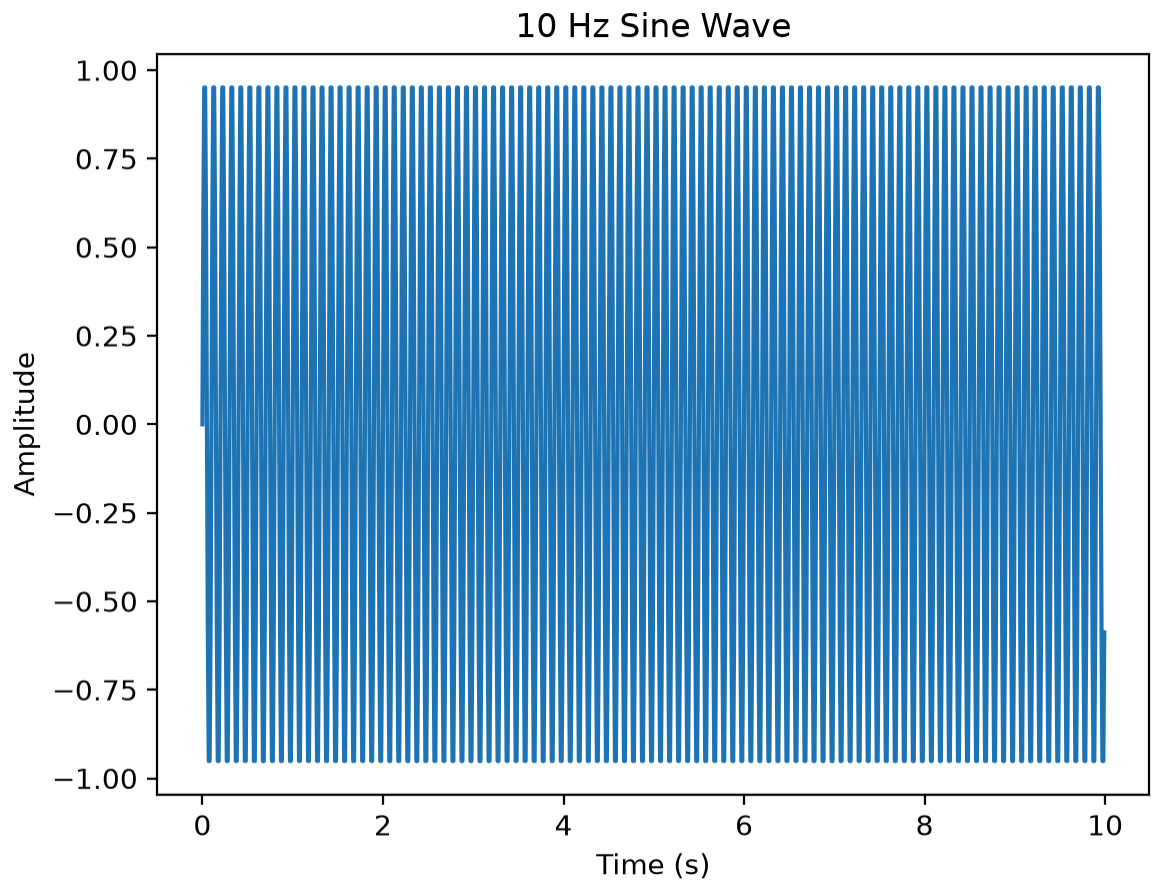
- **Lower** spectral entropy → power is concentrated in a small number of frequencies.
- **Higher** spectral entropy → power is spread across many frequencies.

Before applying this to real EEG, we compare a simple periodic signal with a noisy signal, where the expected direction of the result is known in advance.

Create a simple 10 Hz signal

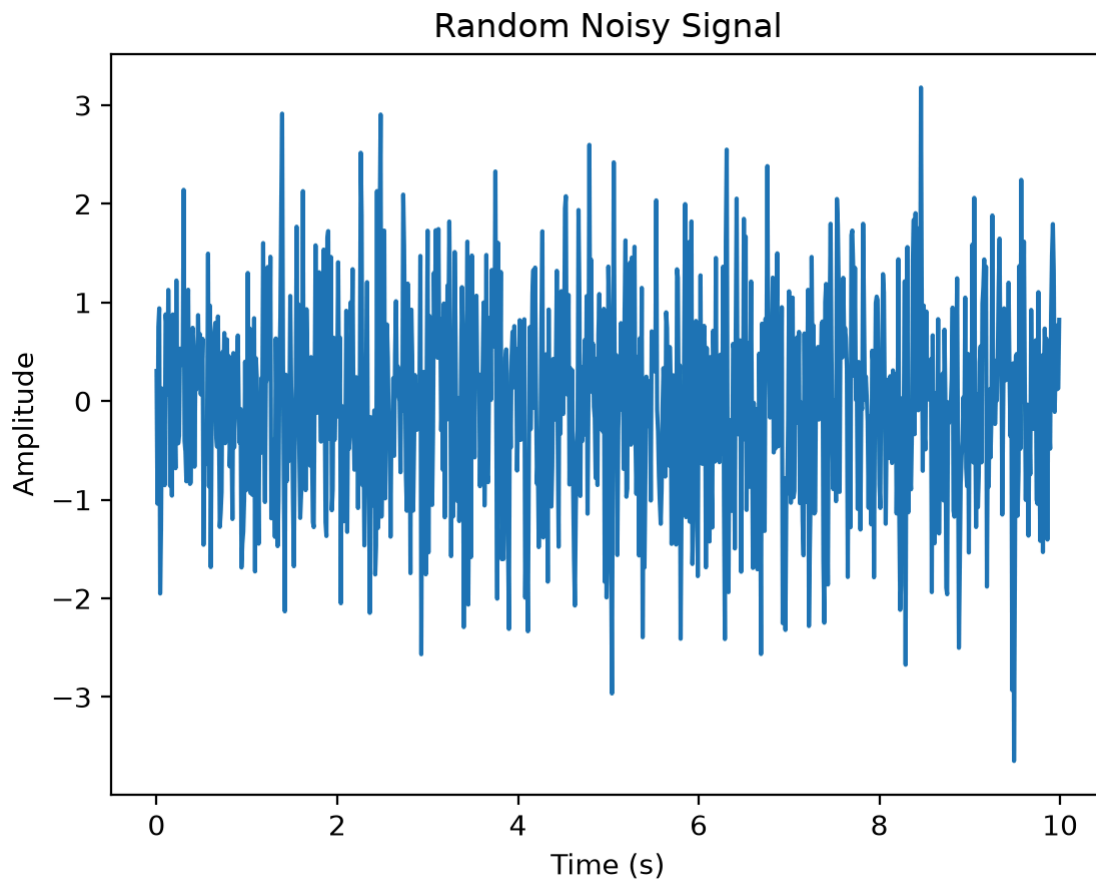
A 10 Hz rhythm falls within the **alpha frequency range** commonly studied in EEG.

This signal is intentionally simple and regular, so its power is concentrated around one frequency.



Create a noisy signal

Next, we create a random noisy signal. Unlike the 10 Hz sine wave, this signal's power is not concentrated at any single frequency, so its spectrum is expected to be broader.



Feature parameters

The important parameters are specified explicitly rather than relying on silently-applied defaults:

- `method="welch"` – estimate the PSD with Welch's method rather than a single periodogram (`'fft'`); Welch averages over multiple overlapping segments, giving a smoother PSD estimate at the cost of frequency resolution.
- `normalize=True` – divide the raw entropy (in bits) by $\log_2(N)$ so the result is bounded to $[0, 1]$ and comparable across signals of different length or sampling rate.
- `nperseg` (not set here) – Antropy leaves this at `None`, which defers to `scipy.signal.welch`'s own default of **256 samples per segment**. This is a hidden-but-real default: at `sf=100 Hz` it corresponds to ~ 2.56 s segments; for the EEG recording later in this notebook (`sf=160 Hz`) it corresponds to ~ 1.6 s segments. Changing `nperseg` changes frequency resolution and can change the resulting entropy value, so it should be reported when comparing results across studies.

Calculate spectral entropy

We now calculate the normalized spectral entropy of both synthetic signals using Antropy.

The 10 Hz sine wave is expected to have **lower** spectral entropy, since most of its power is concentrated around one frequency. The noisy signal is expected to have **higher** spectral entropy, since its power is spread across a wider frequency range.

Numerical output — synthetic signals

Signal	Normalized spectral entropy
10 Hz sine wave	0.178
Random noise	0.983

Each value is a single scalar in $[0, 1]$. As expected, the clean periodic signal has substantially lower entropy than the random noise, confirming the direction of the effect before applying the same feature to real EEG.

Part 2 — Applying spectral entropy to real EEG

After validating the feature's behavior on synthetic signals, we now apply it to a real EEG recording.

Dataset

We use the public **EEG Motor Movement/Imagery Dataset (EEGBCI)** distributed through PhysioNet — the same dataset used in the MNE and YASA examples in this Atlas.

Example recording:

- Subject: 1
- Run: 2
- Condition: baseline, eyes closed
- Recording type: continuous EEG

Run 2 corresponds to the eyes-closed baseline according to the MNE EEGBCI dataset documentation — the same mapping verified in the YASA example.

The data are downloaded automatically by MNE if they are not already available locally.

✓ **General**

Filename(s)	S001R02.edf
MNE object type	RawEDF
Measurement date	2009-08-12 at 16:15:00 UTC
Participant	X
Experimenter	Unknown

✓ **Acquisition**

Duration	00:01:01 (HH:MM:SS)
Sampling frequency	160.00 Hz
Time points	9,760

✓ **Channels**

EEG	64
Head & sensor digitization	Not available

✓ **Filters**

Highpass	0.00 Hz
----------	---------

Recording information

Field	Value
Subject	1
Run	2
Channels	64
Sampling frequency	160.0 Hz
Duration	61.0 seconds

Preparation

Only channel-name standardization is applied. No filtering, resampling, epoching, ICA, or artifact removal is performed, and no montage is attached (not needed without a spatial visualization). This is intentional: the example focuses on the AntroPy feature itself rather than a complete EEG preprocessing pipeline.

Select a channel and compute the feature

For simplicity, we select a single EEG channel and calculate its normalized spectral entropy using the same parameters used for the synthetic signals above.

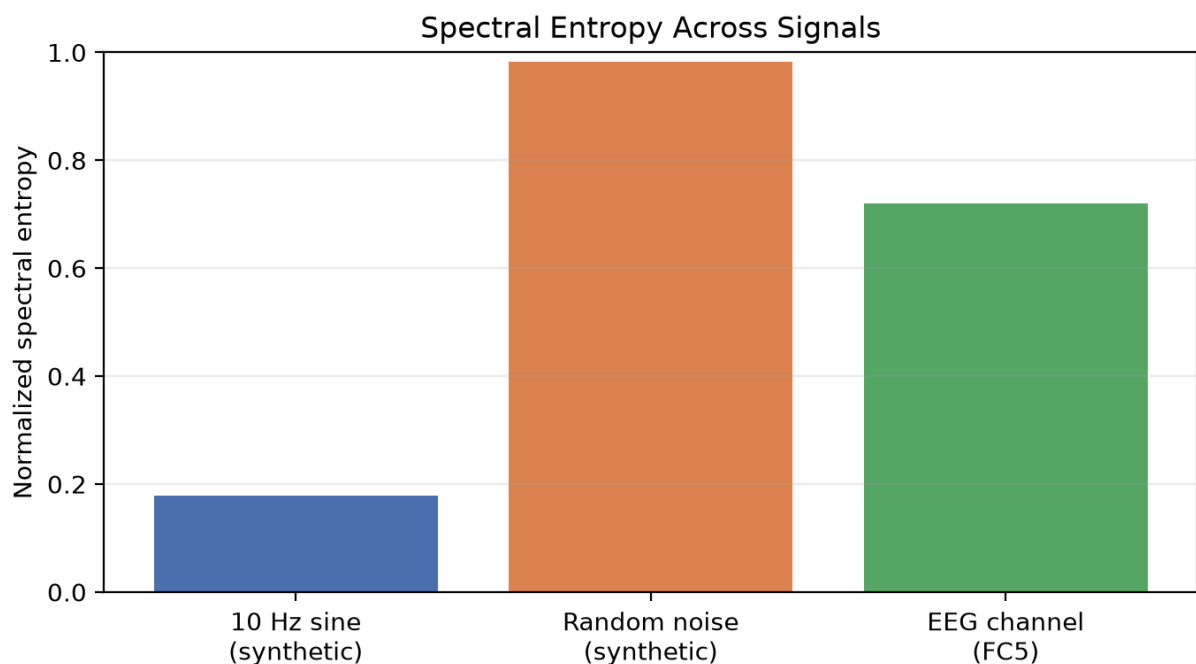
Numerical output — EEG channel

- **Channel:** FC5
- **Normalized spectral entropy:** 0.720

This is a single scalar summarizing how broadly the spectral power of this one EEG channel is distributed across frequencies, using the same definition and parameters applied to the synthetic signals above.

Visualize — comparing all three values

All three spectral entropy values share the same scale ($[0, 1]$, `normalize=True`) and the same estimation method (`method="welch"`), so they can be compared directly in a single bar chart.



Interpretation

Mathematical interpretation

Spectral entropy is the (normalized) Shannon entropy of the signal's power spectral density. A value near 0 means nearly all spectral power sits in a narrow frequency range; a value near 1 means power is close to uniformly distributed across all resolved frequencies. It says nothing about *which* frequencies carry the power, only how concentrated or spread out the distribution is.

EEG interpretation

A single EEG channel's spectral entropy is a coarse summary of that channel's spectral shape at one point in time, and should not by itself be interpreted as evidence of a specific cognitive, neurological, or clinical state. It also depends on parameters such as `method` and `nperseg` (see Feature parameters above) — values are only directly comparable when computed with matching parameters.

This example does not apply filtering or artifact removal to the EEG channel, so broadband noise or artifacts could inflate the measured entropy; a full pipeline would typically band-limit and clean the signal before computing this feature.

Atlas Summary

Field	Value
Library	AntroPy
Method	Shannon entropy of Welch PSD
Feature	Spectral entropy
Domain	Spectral / Complexity
Input	1-D signal (synthetic or single EEG channel)
Output	Scalar, bounded to [0, 1] when <code>normalize=True</code>
Visualization	Bar chart (cross-signal comparison)
Main caveat	Value depends on <code>method</code> and <code>nperseg</code> ; not directly comparable across mismatched parameters
EEG caveat	No filtering/artifact removal applied here; broadband noise can inflate the value

Feature flow

Synthetic signal / EEG channel → AntroPy `spectral_entropy()` → scalar value → Bar chart comparison

This example demonstrates how the same AntroPy feature can be validated on synthetic signals with a known expected direction, then applied unchanged to a real EEG channel.

References

AntroPy

Vallat, R. AntroPy: entropy and complexity of (EEG) time-series in Python. Official documentation and API reference: `spectral_entropy` —

<https://raphaelvallat.com/antropy/>

Scientific reference for spectral entropy

Inouye, T., Shinosaki, K., Sakamoto, H., Toi, S., Ukai, S., Iyama, A., Katsuda, Y., & Hirano, M. (1991). Quantification of EEG irregularity by use of the entropy of the power spectrum. *Electroencephalography and Clinical Neurophysiology*, 79(3), 204–210.

MNE-Python / EEGBCI

MNE-Python documentation: EEGBCI dataset loading and run definitions —

https://mne.tools/stable/generated/mne.datasets.eegbci.load_data.html

Dataset

EEG Motor Movement/Imagery Dataset (EEGBCI), PhysioNet —

<https://physionet.org/content/eegmmidb/1.0.0/>

Scientific reference for EEGBCI

Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., & Wolpaw, J. R. (2004). BCI2000: A General-Purpose Brain-Computer Interface. *IEEE Transactions on Biomedical Engineering*, 51(6), 1034–1043.

No local file paths are used. The EEG dataset is obtained through MNE's public dataset loader.

Reproducibility / Environment

Required Python packages:

- `marimo`
- `mne`
- `numpy`
- `matplotlib`
- `antropy`

The example uses public data and a fixed random seed (`np.random.default_rng(42)`) for the synthetic noisy signal, and contains no hard-coded local dataset paths.

Recommended project configuration:

```
[project]
dependencies = [
    "marimo",
    "mne",
    "numpy",
    "matplotlib",
```

```
    "antropy",  
]
```

The notebook can be launched with:

```
uv run marimo edit <filename>.py
```

or executed as an application with:

```
uv run marimo run <filename>.py
```

Final Checklist

✓ Clear title ✓ Library identified ✓ Method identified ✓ Feature identified ✓ Domain identified ✓ Dataset described ✓ Subject and Run specified ✓ Sampling frequency displayed ✓ Channels described ✓ Preprocessing visible ✓ No hidden transformations (including `nperseg` default) ✓ Main API call visible ✓ Important parameters explained ✓ Numerical output displayed ✓ Output shape explained (scalar, [0, 1]) ✓ Relevant visualization (bar chart) ✓ Axes and units identified ✓ Mathematical interpretation ✓ EEG interpretation with appropriate caution ✓ No unsupported clinical claims ✓ Public dataset ✓ Official documentation identified, with links ✓ Scientific reference identified, with correct citation ✓ Atlas Summary included ✓ No local paths